

وزارة الاتصالات
وتكنولوجيا المعلومات



AI-Powered Predictive Maintenance for Industrial Equipment

Milestone 3: Predictive Maintenance Model Development and Optimization

Agenda

AI-Powered Predictive Maintenance for Industrial Equipment.....	1
Introduction	3
1. RUL Prediction – LSTM Regression Model	4
2. Failure Type Prediction – Multiclass Classification	6
3. Maintenance Required Prediction – Binary Classification	8
Key Challenges	10
Conclusion & Recommendations	11

Introduction

The modeling phase represents the core of the predictive maintenance project. In this stage, multiple machine learning models were built and trained to predict three key aspects related to smart industrial machines:

- **Remaining Useful Life Prediction (RUL Regression)** using an LSTM model.
- **Failure Type Classification** using multiclass models (Random Forest, XGBoost, LightGBM).
- **Maintenance Requirement Prediction** using binary classifiers (Random Forest, XGBoost).
- Some **Challenges** & Some **Models** but was not so good

The modeling process relied on the dataset “**preprocessed_smart_data.csv**” following all cleaning and preprocessing steps. The focus was on handling class imbalance, feature engineering, optimizing hyperparameters, model evaluation, and saving the best performing models for deployment.

Deep learning models were implemented using **PyTorch**, while classical machine learning models were built using **Scikit-learn**, **XGBoost**, and **LightGBM**.

1. RUL Prediction – LSTM Regression Model

Key Steps

1. Feature Engineering

Additional engineered features were created to enhance temporal patterns:

- Moving averages for temperature and vibration (window = 5)
- Rate of change (diff) for temperature and vibration
- Key features used:
Temperature, Vibration, Humidity, Pressure, Energy Consumption, Anomaly Flag, Downtime Risk, Moving Averages, Diff Rates

2. Handling Data Imbalance

Since low RUL values were rare, imbalance was addressed by:

- Selecting a low-RUL threshold (best threshold = **80**)
- Oversampling low-RUL samples $\times 3$
- Adding synthetic noisy samples using Gaussian noise (std = 0.1)

3. Sequence Building

- Time-series sequences were generated using a window size = **30**
- Each sequence contains 30 time steps $\rightarrow$ predicts RUL for the next step

4. Model Training

- **Architecture:** 3-layer LSTM (hidden size = 100, dropout = 0.3)
- **Optimizer:** SGD with Momentum (best = **0.8**)
- **Loss Function:** MAE (L1 Loss)
- **Scheduler:** ReduceLROnPlateau
- **Epochs:** 30–100
- **Batch Size:** 64

5. Hyperparameter Tuning

Grid search experiments were performed on:

- Optimizers (Adam, AdamW, RMSProp, SGD)
- Momentum values (0.7–0.99)
- RUL thresholds (30–80)

Evaluation Metric

- **R² Score** (primary)
- MAE for loss tracking

Results

- **Best Optimizer:** SGD + Momentum = **0.8**
- **Best Threshold:** 80
- **Best R² Score:** ~0.436
- **Best Model Saved As:**
`best_lstm_RUL_M0.8_T80_R2_0.4358.pth`

Challenges

- $R^2 < 0.5$ suggests high noise or insufficient temporal patterns
- Overfitting handled with dropout and LR scheduling, but still present
- LSTM performance depends heavily on data quantity and sequence quality

Insights

- LSTM models are suitable for time-series but computationally expensive
- Iterative tuning improved R² by ~1–2% per training round
- Transformer-based models may achieve higher performance in future iterations

2. Failure Type Prediction – Multiclass Classification

Key Steps

1. Feature Engineering

- Same core features as RUL model
- **Machine ID** encoded using one-hot encoding

2. Handling Class Imbalance

- Severe imbalance across failure types
- **SMOTE** used to generate synthetic samples for minority classes

3. Model Training

Models trained with `n_estimators = 300`:

- **Random Forest**
- **XGBoost** (objective = "multi:softprob", metric = "mlogloss")
- **LightGBM**

4. Hyperparameter Optimization

- **RandomizedSearchCV** (`n_iter = 10`, `CV = 3`)
- **Tuned parameters:**
`n_estimators`, `max_depth`, `learning_rate`, `subsample`, etc.

Evaluation Metrics

- Accuracy
- F1-Score (Macro / Weighted)
- Confusion Matrix
- Classification Report

Results

- **Best Performing Models:** XGBoost or LightGBM (depending on tuning round)
- **Accuracy:** ~ 0.92
- **F1-Macro:** ~ 0.75 (lower due to rare classes)
- **F1-Weighted:** ~ 0.90
- Saved Model:
best_model_failure_type.pkl

Insights

- Normal failure type dominates the dataset $\rightarrow$ model biased without SMOTE
- SMOTE significantly improved recall for rare failure types
- LightGBM provided the fastest training time
- XGBoost achieved the most balanced performance

3. Maintenance Required Prediction – Binary Classification

Key Steps

1. Feature Engineering

Same as previous tasks, emphasizing:

- **Anomaly Flag**
- **Downtime Risk**

2. Model Training

- **Random Forest** (class_weight = 'balanced')
- **XGBoost** (eval_metric = 'logloss')

3. Hyperparameter Tuning

- RandomizedSearchCV on:
n_estimators, max_depth, learning_rate, subsample, colsample_bytree

Evaluation Metrics

- Accuracy
- F1-Score
- ROC-AUC
- Confusion Matrix

Results

- **Best Model:** XGBoost
- **Accuracy:** ~0.90
- **F1-Score:** ~0.85
- **ROC-AUC:** ~0.94

- Saved Model:
best_model_machine_fail.pkl

Insights

- Binary classification is easier than multiclass → higher performance
- Random Forest handled noise well, but XGBoost outperformed after tuning
- Strong ROC-AUC shows excellent separation between failure and non-failure states

Key Challenges

During the modeling process, several challenges emerged due to the nature of the data, the complexity of the engineered features, and the behavior of the tested models (LSTM, multi-output deep learning, and XGBoost). These challenges are summarized as follows:

1. Data Quality & Structure Challenges

- **High noise and fluctuations** in sensor readings (temperature, vibration, pressure) caused unstable behavior in time-series models.
- **Extensive missing values** generated by lag, rolling, and rate-change features required heavy cleaning and reduced usable data.
- **Severe class imbalance**, especially for failure labels (≤ 5 , ≤ 10 , ≤ 15 minutes), made it difficult for models to learn rare failure patterns.

2. Modeling Challenges

- **LSTM instability:**
 - Sensitive to window size, scaling, and noise.
 - Frequent overfitting despite dropout and gradient clipping.
 - Achieved lower R^2 compared to tree-based models.
- **Multi-output deep learning** (RUL + 5/10/15 min failures) suffered from:
 - Conflicting objectives between regression and classification.
 - Difficulty in balancing multi-task loss, leading to inconsistent performance across outputs.
- **XGBoost complexity:**
 - Large number of engineered features increased depth, training time, and computational cost.
 - Required careful tuning of `scale_pos_weight` to mitigate imbalance.

3. Short-Term Failure Prediction Challenges (5/10/15 Minutes)

- **Very few positive samples**, especially for ≤ 5 minutes, leading to low recall and unstable AUC.
- **Concept drift** between machines and time periods made failure patterns inconsistent.
- **Label sensitivity**: small timestamp shifts could change a failure from the 5-minute class to the 10-minute class, increasing noise in classification labels.

Conclusion & Recommendations

Overall Performance

- LSTM RUL Model: $R^2 \approx 0.44$
- Failure Type Classification: **F1-Macro** $\sim 0.85-0.95$
- Maintenance Required Classification: **ROC-AUC** ~ 0.94

The models show strong performance, but data imbalance and limited time-series depth remain significant challenges.

Recommendations

1. Enhance Model Performance

- Adopt **Ensemble Stacking** to combine RF + XGB + LGBM
- Experiment with **Transformer-based architectures** for RUL
- Apply **k-fold cross-validation** to improve robustness
- Increase sequence length or incorporate multi-step forecasting

2. Improve Dataset Quality

- Collect longer time-series histories
- Increase failure event samples
- Add more operational context (load, weather, cycles, etc.)

3. Deployment Preparation

- Export models into a unified API (FastAPI)
- Build a real-time monitoring dashboard
- Implement automatic model retraining pipelines

4. Deployment & Generalization Challenges

- Sequence models require **continuous and clean real-time data**, making them hard to deploy in environments with missing timestamps.
- Predictions could become **logically inconsistent** (e.g., RUL high but failure-in-10-min predicted positive), requiring additional post-processing logic.

Summary

Overall, the main difficulties were driven by:

- Noisy time-series data
- Heavy imbalance in failure events
- LSTM sensitivity and overfitting
- Complexity of multi-task modeling
- Sparse short-term failure labels
- High computational cost for engineered features

Despite these challenges, XGBoost delivered more stable performance, while LSTM models offered deeper temporal insights but required more data and tuning.